

DBMS Lab: SQL Joins and Transactions

Monish Putti

2520030395

Part A: Create Tables

```
CREATE TABLE roles(role_id INT PRIMARY KEY, role_name
VARCHAR(50));
CREATE TABLE users(user_id INT PRIMARY KEY,name
VARCHAR(50),email VARCHAR(50),role_id INT);
CREATE TABLE departments(dept_id INT PRIMARY KEY,dept_name
VARCHAR(50));
CREATE TABLE employees(emp_id INT PRIMARY KEY,emp_name
VARCHAR(50),dept_id INT,manager_id INT,salary DECIMAL(10,2));
CREATE TABLE customers(customer_id INT PRIMARY
KEY,customer_name VARCHAR(50),city VARCHAR(50));
CREATE TABLE orders(order_id INT PRIMARY KEY,customer_id
INT,order_date DATE,total_amount DECIMAL(10,2));
CREATE TABLE accounts(account_no INT PRIMARY KEY,holder_name
VARCHAR(50),balance DECIMAL(10,2));
```

```
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 23
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE dbms_lab;
Query OK, 1 row affected (0.01 sec)

mysql> USE dbms_lab;
Database changed
mysql> CREATE TABLE roles(
  ->     role_id INT PRIMARY KEY,
  ->     role_name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE users(
  ->     user_id INT PRIMARY KEY,
  ->     name VARCHAR(50),
  ->     email VARCHAR(50),
  ->     role_id INT
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE departments(
  ->     dept_id INT PRIMARY KEY,
  ->     dept_name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE employees(
  ->     emp_id INT PRIMARY KEY,
  ->     emp_name VARCHAR(50),
```

```
->     emp_name VARCHAR(50),
->     dept_id INT,
->     manager_id INT,
->     salary DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE customers(
->     customer_id INT PRIMARY KEY,
->     customer_name VARCHAR(50),
->     city VARCHAR(50)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE orders(
->     order_id INT PRIMARY KEY,
->     customer_id INT,
->     order_date DATE,
->     total_amount DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE accounts(
->     account_no INT PRIMARY KEY,
->     holder_name VARCHAR(50),
->     balance DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO roles VALUES
-> (1,'Admin'),
-> (2,'Faculty'),
-> (3,'Student'),
-> (4,'Guest');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO users VALUES
-> (101,'Amit','amit@gmail.com',1),
-> (102,'Priya','priya@gmail.com',2),
-> (103,'Rahul','rahul@gmail.com',3);
```

```
-> (101,'Amit','amit@gmail.com',1),
-> (102,'Priya','priya@gmail.com',2),
-> (103,'Rahul','rahul@gmail.com',3);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO departments VALUES
-> (1,'CSE'),
-> (2,'ECE'),
-> (3,'MECH');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO employees VALUES
-> (1,'Raj',1,NULL,80000),
-> (2,'Neha',1,1,60000),
-> (3,'Arun',2,1,50000),
-> (4,'Kiran',3,1,45000);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO customers VALUES
-> (1,'John','Delhi'),
-> (2,'Alice','Mumbai'),
-> (3,'David','Chennai');
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO orders VALUES
-> (101,1,'2025-07-01',1500),
-> (102,1,'2025-07-15',3000),
-> (103,2,'2025-07-20',2500),
-> (104,3,'2025-07-25',5000);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO accounts VALUES
-> (101,'Ramesh',50000),
-> (102,'Suresh',30000),
-> (103,'Mahesh',45000);
Query OK, 3 rows affected (0.00 sec)
```

```
-> (102,'Suresh',30000),
-> (103,'Mahesh',45000);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SHOW TABLES;
```

Tables_in_dbms_lab
accounts
customers
departments
employees
orders
roles
users

```
7 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM roles;
```

role_id	role_name
1	Admin
2	Faculty
3	Student
4	Guest

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM users;
```

user_id	name	email	role_id
101	Amit	amit@gmail.com	1
102	Priya	priya@gmail.com	2
103	Rahul	rahul@gmail.com	3

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM departments;
```

```
mysql> SELECT * FROM departments;
```

dept_id	dept_name
1	CSE
2	ECE
3	MECH

3 rows in set (0.00 sec)

```
mysql> SELECT * FROM employees;
```

emp_id	emp_name	dept_id	manager_id	salary
1	Raj	1	NULL	80000.00
2	Neha	1	1	60000.00
3	Arun	2	1	50000.00
4	Kiran	3	1	45000.00

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM customers;
```

customer_id	customer_name	city
1	John	Delhi
2	Alice	Mumbai
3	David	Chennai

3 rows in set (0.00 sec)

```
mysql> SELECT * FROM orders;
```

order_id	customer_id	order_date	total_amount
101	1	2025-07-01	1500.00
102	1	2025-07-15	3000.00
103	2	2025-07-20	2500.00
104	3	2025-07-25	5000.00

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM accounts;mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
| 101 | Ramesh | 50000.00 |
| 102 | Suresh | 30000.00 |
| 103 | Mahesh | 45000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'n
ysql> SELECT * FROM accounts' at line 1
mysql> SELECT * FROM accounts;mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
| 101 | Ramesh | 50000.00 |
| 102 | Suresh | 30000.00 |
| 103 | Mahesh | 45000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'n
ysql> SELECT * FROM accounts' at line 1
mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
| 101 | Ramesh | 50000.00 |
| 102 | Suresh | 30000.00 |
| 103 | Mahesh | 45000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

Q1. Display employee names with their department names using INNER JOIN.

SELECT emp_name,dept_name FROM employees e INNER JOIN departments d ON
e.dept_id=d.dept_id;

```
mysql> SELECT emp_name, dept_name
-> FROM employees e
-> INNER JOIN departments d
-> ON e.dept_id = d.dept_id;
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Raj      | CSE       |
| Neha     | CSE       |
| Arun     | ECE       |
| Kiran    | MECH      |
+-----+-----+
4 rows in set (0.00 sec)

mysql> |
```

Q2. Display all users with their assigned roles using LEFT JOIN.

SELECT name,email,role_name FROM users u LEFT JOIN roles r ON
u.role_id=r.role_id;

```
mysql> SELECT name, email, role_name
-> FROM users u
-> LEFT JOIN roles r
-> ON u.role_id = r.role_id;
+-----+-----+-----+
| name | email | role_name |
+-----+-----+-----+
| Amit | amit@gmail.com | Admin |
| Priya | priya@gmail.com | Faculty |
| Rahul | rahul@gmail.com | Student |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

Q3. Display all roles even if no users are assigned using RIGHT JOIN.

SELECT name,role_name FROM users u RIGHT JOIN roles r ON u.role_id=r.role_id;

```
mysql> SELECT name, role_name
-> FROM users u
-> RIGHT JOIN roles r
-> ON u.role_id = r.role_id;
+-----+-----+
| name | role_name |
+-----+-----+
| Amit | Admin |
| Priya | Faculty |
| Rahul | Student |
| NULL | Guest |
+-----+-----+
4 rows in set (0.00 sec)

mysql> |
```

Q4. Simulate FULL OUTER JOIN in MySQL.

SELECT name,email,role_name FROM users u LEFT JOIN roles r ON
u.role_id=r.role_id UNION SELECT name,email,role_name FROM users u RIGHT
JOIN roles r ON u.role_id=r.role_id;

```
mysql> SELECT name, email, role_name
-> FROM users u
-> LEFT JOIN roles r
-> ON u.role_id = r.role_id
->
-> UNION
->
-> SELECT name, email, role_name
-> FROM users u
-> RIGHT JOIN roles r
-> ON u.role_id = r.role_id;
```

name	email	role_name
Amit	amit@gmail.com	Admin
Priya	priya@gmail.com	Faculty
Rahul	rahul@gmail.com	Student
NULL	NULL	Guest

4 rows in set (0.00 sec)

```
mysql> |
```

Q5. Display employee and manager names using SELF JOIN.

SELECT e.emp_name Employee,m.emp_name Manager FROM employees e LEFT
JOIN employees m ON e.manager_id=m.emp_id;

```
mysql> SELECT e.emp_name AS Employee,
->          m.emp_name AS Manager
-> FROM employees e
-> LEFT JOIN employees m
-> ON e.manager_id = m.emp_id;
```

Employee	Manager
Raj	NULL
Neha	Raj
Arun	Raj
Kiran	Raj

```
4 rows in set (0.00 sec)

mysql> |
```

Q6. Display Order ID, Customer Name, City and Total Amount.

SELECT o.order_id,customer_name,city,total_amount FROM customers c JOIN orders o
ON c.customer_id=o.customer_id;

```
mysql> SELECT o.order_id,
->          customer_name,
->          city,
->          total_amount
-> FROM customers c
-> JOIN orders o
-> ON c.customer_id = o.customer_id;
```

order_id	customer_name	city	total_amount
101	John	Delhi	1500.00
102	John	Delhi	3000.00
103	Alice	Mumbai	2500.00
104	David	Chennai	5000.00

```
4 rows in set (0.00 sec)

mysql> |
```

Q7. Find department-wise employee count, average salary and maximum salary.

```
SELECT dept_name,COUNT(*),AVG(salary),MAX(salary) FROM employees e JOIN  
departments d ON e.dept_id=d.dept_id GROUP BY dept_name;
```

```
mysql> SELECT dept_name,  
-> COUNT(*) AS Employee_Count,  
-> AVG(salary) AS Average_Salary,  
-> MAX(salary) AS Maximum_Salary  
-> FROM employees e  
-> JOIN departments d  
-> ON e.dept_id = d.dept_id  
-> GROUP BY dept_name;
```

dept_name	Employee_Count	Average_Salary	Maximum_Salary
CSE	2	70000.000000	80000.00
ECE	1	50000.000000	50000.00
MECH	1	45000.000000	45000.00

```
3 rows in set (0.01 sec)  
  
mysql> |
```

Q8. Display employees earning >50000 in CSE.

```
SELECT emp_name,salary,dept_name FROM employees e JOIN departments d ON  
e.dept_id=d.dept_id WHERE salary>50000 AND dept_name='CSE';
```

```
mysql> SELECT emp_name, salary, dept_name  
-> FROM employees e  
-> JOIN departments d  
-> ON e.dept_id = d.dept_id  
-> WHERE salary > 50000  
-> AND dept_name = 'CSE';
```

emp_name	salary	dept_name
Raj	80000.00	CSE
Neha	60000.00	CSE

```
2 rows in set (0.00 sec)  
  
mysql> |
```

Q9. Display customer-wise total purchase in descending order.

SELECT customer_name,SUM(total_amount) TotalPurchase FROM customers c JOIN orders o ON c.customer_id=o.customer_id GROUP BY customer_name ORDER BY TotalPurchase DESC;

```
mysql> SELECT customer_name,
->          SUM(total_amount) AS TotalPurchase
-> FROM customers c
-> JOIN orders o
-> ON c.customer_id = o.customer_id
-> GROUP BY customer_name
-> ORDER BY TotalPurchase DESC;
+-----+-----+
| customer_name | TotalPurchase |
+-----+-----+
| David         | 5000.00      |
| John          | 4500.00      |
| Alice         | 2500.00      |
+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

Q10. Create Students, Courses, Faculty, Enrollments and display Student, Course, Faculty using joins.

SELECT student_name,course_name,faculty_name FROM students s JOIN enrollments e ON s.student_id=e.student_id JOIN courses c ON e.course_id=c.course_id JOIN faculty f ON c.faculty_id=f.faculty_id;

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> USE dbms_lab;
Database changed
mysql> CREATE TABLE students (
  -> student_id INT PRIMARY KEY,
  -> student_name VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'students' already exists
mysql> CREATE TABLE faculty (
  -> faculty_id INT PRIMARY KEY,
  -> faculty_name VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'faculty' already exists
mysql>
mysql> CREATE TABLE courses (
  -> course_id INT PRIMARY KEY,
  -> course_name VARCHAR(50),
  -> faculty_id INT
  -> );
ERROR 1050 (42S01): Table 'courses' already exists
mysql>
mysql> CREATE TABLE enrollments (
  -> student_id INT,
  -> course_id INT
  -> );
ERROR 1050 (42S01): Table 'enrollments' already exists
```

```
mysql> SELECT student_name, course_name, faculty_name
  -> FROM students s
  -> JOIN enrollments e
  -> ON s.student_id = e.student_id
  -> JOIN courses c
  -> ON e.course_id = c.course_id
  -> JOIN faculty f
  -> ON c.faculty_id = f.faculty_id;
```

student_name	course_name	faculty_name
Rahul	DBMS	Dr. Kumar
Priya	Java	Dr. Sharma
Rahul	DBMS	Dr. Kumar
Priya	Java	Dr. Sharma

4 rows in set (0.02 sec)

```
mysql> |
```

```

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO accounts
    -> VALUES (104, 'Anil', 25000);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
|          101 | Ramesh      | 50000.00 |
|          102 | Suresh      | 30000.00 |
|          103 | Mahesh      | 45000.00 |
|          104 | Anil        | 25000.00 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |

```

Part B: Transactions

Q11. Insert a new account and COMMIT.

START TRANSACTION; INSERT INTO accounts VALUES(104,'Anil',25000);
COMMIT;

```

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO accounts
    -> VALUES (104, 'Anil', 25000);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
|          101 | Ramesh      | 50000.00 |
|          102 | Suresh      | 30000.00 |
|          103 | Mahesh      | 45000.00 |
|          104 | Anil        | 25000.00 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |

```

Q12. Insert a record and ROLLBACK.

START TRANSACTION; INSERT INTO accounts VALUES(105,'Ravi',10000);
 ROLLBACK;

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO accounts
    -> VALUES (105, 'Ravi', 10000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
|          101 | Ramesh      | 50000.00 |
|          102 | Suresh      | 30000.00 |
|          103 | Mahesh      | 45000.00 |
|          104 | Anil        | 25000.00 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |
```

Q13. Transfer ₹5000 from Account 101 to 102.

START TRANSACTION; UPDATE accounts SET balance=balance-5000 WHERE account_no=101; UPDATE accounts SET balance=balance+5000 WHERE account_no=102; COMMIT;

```

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
      -> SET balance = balance - 5000
      -> WHERE account_no = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE accounts
      -> SET balance = balance + 5000
      -> WHERE account_no = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
|          101 | Ramesh      | 45000.00 |
|          102 | Suresh      | 35000.00 |
|          103 | Mahesh      | 45000.00 |
|          104 | Anil        | 25000.00 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |

```

Q14. Simulate failed transaction and rollback.

START TRANSACTION; UPDATE accounts SET balance=balance-3000 WHERE account_no=101; UPDATE accounts SET balance=balance+3000 WHERE account_no=999; ROLLBACK;

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
    -> SET balance = balance - 3000
    -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE accounts
    -> SET balance = balance + 3000
    -> WHERE account_no = 999;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
|          101 | Ramesh      | 45000.00 |
|          102 | Suresh      | 35000.00 |
|          103 | Mahesh      | 45000.00 |
|          104 | Anil        | 25000.00 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |
```

Q15. Demonstrate SAVEPOINT.

```
START TRANSACTION; UPDATE accounts SET balance=balance+2000 WHERE  
account_no=101; SAVEPOINT s1; UPDATE accounts SET balance=balance-1000  
WHERE account_no=101; ROLLBACK TO s1; COMMIT;
```

```
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>  
mysql> UPDATE accounts  
    -> SET balance = balance + 2000  
    -> WHERE account_no = 101;  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql>  
mysql> SAVEPOINT s1;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>  
mysql> UPDATE accounts  
    -> SET balance = balance - 1000  
    -> WHERE account_no = 101;  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql>  
mysql> ROLLBACK TO s1;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>  
mysql> COMMIT;  
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM accounts;  
+-----+-----+-----+  
| account_no | holder_name | balance |  
+-----+-----+-----+  
|          101 | Ramesh      | 47000.00 |  
|          102 | Suresh      | 35000.00 |  
|          103 | Mahesh      | 45000.00 |  
|          104 | Anil        | 25000.00 |  
+-----+-----+-----+  
4 rows in set (0.00 sec)
```